ServerPack
Cloud Tenant Virtual Machine Allocatorcator

Name: Manan Jain

Roll Number: 150096725166

Institution: ITM Skills University

Subject: React Development

Batch: 2025-29

GitHub:https://github.com/mananjain20/virtual-machine-allocator-dsa

Problem Statement

ServerPack is the management platform for a massive data center that assigns virtual machines to customers, monitors hardware utilization, and optimizes power consumption. The existing system has several critical problems:

- Looking up physical server resource stats is inconsistent and slow
- Incorrect VM resizing operations cannot be rolled back
- Provisioning requests arrive in bulk but get handled randomly
- The system cannot rank servers by current utilization — some machines run at full capacity while others are nearly idle
- No way to consolidate workloads during off-peak hours to save power

Objectives

1. Maintain an organized hardware inventory with fast O(1) server lookups

2. Support rollback of incorrect VM resize operations using a stack
3. Process VM provisioning requests in strict FIFO order
4. Verify VM operating systems against a sorted valid OS list
5. Rank physical servers by utilization for load balancing
6. Model datacenter network connectivity as a weighted graph
7. Calculate minimum latency path between two servers using Dijkstra's algorithm
8. Consolidate workloads onto the fewest servers to maximize power savings

Design / Architecture

The system is built around a single core class CloudManager that manages all operations. The design follows a modular approach where each feature is handled by an appropriate data structure.

Core Components

• **CloudManager** — Central controller class managing all 8 system features

• **Server** — Represents a physical machine with CPU, RAM and list of assigned VMs

• **VM** — Represents a virtual machine with ID, operating system and RAM

• **ResizeHistory** — Stores the previous RAM value for undo/rollback operations

Key Modules

• **Inventory Module** — Add and view physical servers

• **Provisioning Module** — Queue and assign VMs to servers in FIFO order

• **Resize Module** — Resize VMs with full undo support

• **Verification Module** — Check OS against a valid sorted list

• **Network Module** — Map server connections and find the fastest path

• **Optimization Module** — Sort servers by load and recommend power savings

Algorithm Description

1. Hardware Inventory — unordered_map

Servers are stored in an unordered_map keyed by server ID. This provides O(1) average-case
access for add and lookup operations. This mirrors how VMware vCenter maintains a
hash-indexed host table.

2. Resize Undo — Stack

Every resize operation pushes the previous RAM value onto a stack. Calling undo pops the top
entry and restores the VM to its original configuration. LIFO behavior ensures the most recent
resize is always undone first.

3. Provisioning Queue — Queue

VM requests are pushed into a queue and processed in strict FIFO order. This ensures fairness
across all provisioning requests, similar to RabbitMQ task queues used in OpenStack Nova.

4. OS Verification — Binary Search

A sorted vector of valid operating systems is searched using binary_search, giving O(log n)
verification time. The list is sorted once and reused for all verification calls.

5. Density Sorter — std::sort

Servers are copied into a temporary vector and sorted in descending order by VM count using
std::sort with a lambda comparator. This directly supports load balancing and utilization
Visibility.

6. Datacenter Map — Adjacency List

The network is modeled as an undirected weighted graph using an adjacency list. Each server
is a node and each physical connection is a weighted edge representing latency in
Milliseconds.

7. Fastest Connection — Dijkstra's Algorithm

A min-heap based priority queue implements Dijkstra's algorithm to find the minimum latency
path between any two servers. This is the same approach used by SDN controllers for optimal
Routing.

8. Power Saver — Greedy Sort

Servers are sorted by utilization. Underloaded servers (1 or fewer VMs) are flagged with a
recommendation to migrate workloads to the most loaded server, enabling those underloaded
servers to be powered off.

Complexity Analysis

Complexity Analysis

1. Add / Lookup Server — Time: O(1) average — Space: O(n)
2. Add / Lookup VM — Time: O(1) average — Space: O(m)
3. Enqueue VM Request — Time: O(1) — Space: O(k)
4. Process VM Request — Time: O(n) — Space: O(1)
5. Resize VM — Time: O(1) — Space: O(r)
6. Undo Resize — Time: O(1) — Space: O(1)
7. OS Verification — Time: O(log n) — Space: O(s)
8. Density Sorter — Time: O(n log n) — Space: O(n)
9. Add Graph Connection — Time: O(1) — Space: O(E)
10. Shortest Path Dijkstra — Time: O((V+E) log V) — Space: O(V+E)
11. Power Saver — Time: O(n log n) — Space: O(n)

Note: n = servers, m = VMs, k = queue size, r = resize history, s = OS list size, V = vertices, E = edges

Screenshots of Execution

```cpp
#include <iostream>
#include <unordered_map>
#include <vector>
#include <queue>
#include <stack>
#include <algorithm>
#include <limits>
using namespace std;
// Simple VM record with id, OS and RAM
class VM
{
public:
    string vmId;
    string os;
    int ram;

    VM() {}
    VM(string id, string o, int r) : vmId(id), os(o), ram(r) {}
};

// Simple server record holding resources and hosted VMs
class Server
{
public:
    string serverId;
    int cpu;
    int ram;
    vector<string> vmList;

    Server() {}
    Server(string id, int c, int r) : serverId(id), cpu(c), ram(r) {}
};

// History entry to allow undoing a VM resize
struct ResizeHistory
{
    string vmId;
    int oldRam;
};

// Cloud manager that keeps servers, VMs, network and operations
class CloudManager
{
private:
    // Map serverId -> Server for fast lookup
    unordered_map<string, Server> servers;
    // Map vmId -> VM for fast lookup
    unordered_map<string, VM> vms;

    // Queue for incoming VM requests (FIFO)
    queue<VM> requestQueue;
    // Stack to record resize operations for undo (LIFO)
    stack<ResizeHistory> resizeStack;

    // Allowed OS names (kept in a vector for easy sorting/search)
    vector<string> validOS = {"UBUNTU", "WINDOWS", "LINUX", "DEBIAN", "FEDORA", "MACOS"};

    // Network graph as adjacency list: server -> list of (neighbor, latency)
    unordered_map<string, vector<pair<string, int>>> graph;

public:
    // Add a new server with user-provided specs
    void addServer()
    {
        string id;
        int cpu, ram;

        cout << "Server ID: ";
        cin >> id;
        cout << "CPU Cores: ";
        cin >> cpu;
        cout << "RAM (GB): ";
        cin >> ram;

        servers[id] = Server(id, cpu, ram);
        cout << "Server added.\n";
    }

    // Print all servers and their basic info
    void viewServers()
    {
        if (servers.empty())
        {
            cout << "No servers found.\n";
            return;
        }

        for (auto &entry : servers)
        {
            Server &s = entry.second;
            cout << "\nServer: " << s.serverId
                 << "\nCPU: " << s.cpu
                 << "\nRAM: " << s.ram
                 << "\nVM Count: " << s.vmList.size()
                 << "\n----------------\n";
        }
    }
    // Queue a VM creation request from user input
    void addVMRequest()
    {
        string id, os;
        int ram;
```

```cpp
// Queue a VM creation request from user input
void addVMRequest()
{
    string id, os;
    int ram;

    cout << "VM ID: ";
    cin >> id;
    cout << "OS: ";
    cin >> os;
    cout << "RAM (GB): ";
    cin >> ram;

    requestQueue.push(VM(id, os, ram));
    cout << "Request added to queue.\n";
}

// Assign next queued VM to the least-loaded server
void processVMRequest()
{
    if (requestQueue.empty())
    {
        cout << "No pending requests.\n";
        return;
    }

    if (servers.empty())
    {
        cout << "Add a server first.\n";
        return;
    }

    VM vm = requestQueue.front();
    requestQueue.pop();
    // Find server with minimum VM count
    auto bestServer = servers.begin();

    for (auto it = servers.begin(); it != servers.end(); it++)
    {
        if (it->second.vmList.size() <
            bestServer->second.vmList.size())
        {
            bestServer = it;
        }
    }

    bestServer->second.vmList.push_back(vm.vmId);

    vms[vm.vmId] = vm;

    cout << "VM " << vm.vmId
         << " assigned to Server "
         << bestServer->second.serverId
         << "\n";
}

// List all VMs with their properties
void viewVMs()
{
    if (vms.empty())
    {
        cout << "No VMs available.\n";
        return;
    }

    for (auto &entry : vms)
    {
        VM &vm = entry.second;

        cout << "\nVM ID: " << vm.vmId
             << "\nOS: " << vm.os
             << "\nRAM: " << vm.ram << " GB"
             << "\n----------------\n";
    }
}
// Change RAM of a VM and record history for undo
void resizeVM()
{
    string id;
    int newRam;

    cout << "VM ID: ";
    cin >> id;

    if (vms.find(id) == vms.end())
    {
        cout << "VM not found.\n";
        return;
    }

    cout << "New RAM: ";
    cin >> newRam;

    resizeStack.push({id, vms[id].ram});
    vms[id].ram = newRam;

    cout << "VM resized.\n";
}

```

```cpp
        vms[id].ram = newRam;

        cout << "VM resized.\n";
    }

    // Revert the last VM resize using the resize stack
    void undoResize()
    {
        if (resizeStack.empty())
        {
            cout << "Nothing to undo.\n";
            return;
        }

        ResizeHistory last = resizeStack.top();
        resizeStack.pop();

        vms[last.vmId].ram = last.oldRam;

        cout << "Undo successful.\n";
    }

    // Verify whether an OS is in the allowed list
    void systemCheck()
    {
        // sort OS list once so we can binary search it quickly
        sort(validOS.begin(), validOS.end());

        string os;
        cout << "Enter OS to verify: ";
        cin >> os;

        if (binary_search(validOS.begin(), validOS.end(), os))
            cout << "OS verified.\n";
        else
            cout << "OS not found.\n";
    }

    // Show servers ordered by number of VMs (high -> low)
    void densitySorter()
    {
        vector<Server> list;

        for (auto &entry : servers)
            list.push_back(entry.second);

        // sort servers by VM count (descending) to show utilization
        sort(list.begin(), list.end(),
             [](Server a, Server b)
             {
                 return a.vmList.size() > b.vmList.size();
             });

        cout << "\nServers by utilization:\n";

        for (auto &s : list)
        {
            cout << s.serverId
                 << " -> " << s.vmList.size()
                 << " VMs\n";
        }
    }

    // Add a bidirectional network link between two servers
    void addConnection()
    {
        string a, b;
        int latency;

        cout << "Server A: ";
        cin >> a;
        cout << "Server B: ";
        cin >> b;
        cout << "Latency: ";
        cin >> latency;

        graph[a].push_back({b, latency});
        graph[b].push_back({a, latency});

        cout << "Connection added.\n";
    }
    // Print adjacency list of the datacenter network
    void showMap()
    {
        for (auto &entry : graph)
        {
            cout << "\n"
                 << entry.first << " -> ";

            for (auto &edge : entry.second)
            {
                cout << "(" << edge.first
                     << ", " << edge.second
                     << "ms) ";
            }
        }
        cout << "\n";
    }

    // Compute minimum-latency path between two servers (Dijkstra)
```

```cpp
    // Compute minimum-latency path between two servers (Dijkstra)
    void fastestConnection()
    {
        string source, destination;

        cout << "Source Server: ";
        cin >> source;

        cout << "Destination Server: ";
        cin >> destination;

        unordered_map<string, int> dist;

        for (auto &node : graph)
            dist[node.first] = INT_MAX;

        if (graph.find(source) == graph.end())
        {
            cout << "Source not found.\n";
            return;
        }

        priority_queue<pair<int, string>,
                       vector<pair<int, string>>,
                       greater<pair<int, string>>>
            pq;

        dist[source] = 0;
        pq.push({0, source});

        // Dijkstra main loop: relax edges until all shortest paths found
        while (!pq.empty())
        {
            auto current = pq.top();
            pq.pop();

            int currentDist = current.first;
            string currentNode = current.second;

            for (auto &neighbor : graph[currentNode])
            {
                string nextNode = neighbor.first;
                int weight = neighbor.second;

                if (currentDist + weight < dist[nextNode])
                {
                    dist[nextNode] = currentDist + weight;
                    pq.push({dist[nextNode], nextNode});
                }
            }
        }
        if (dist[destination] == INT_MAX)
            cout << "No path found.\n";
        else
            cout << "Minimum latency = "
                 << dist[destination]
                 << " ms\n";
    }
    // Recommend servers to power off by finding lightly used ones
    void powerSaver()
    {
        vector<Server> list;

        for (auto &entry : servers)
            list.push_back(entry.second);

        // sort servers by VM count (descending) to find light ones
        sort(list.begin(), list.end(),
             [](Server a, Server b)
             {
                 return a.vmList.size() > b.vmList.size();
             });
        cout << "\nPower Saver Recommendation\n";

        for (auto &s : list)
        {
            if (s.vmList.size() <= 1)
            {
                        cout<< "Server " << s.serverId
                        << " (" << s.vmList.size() << " VM) → move to Server "
                        << list[0].serverId
                        << " | Power off " << s.serverId << " to save power.\n";
            }
        }
    }
};
// Menu-driven main to interact with the CloudManager
int main()
{
    CloudManager manager;
    int choice;

    do
    {
        cout << "\n===== SERVERPACK CLOUD SYSTEM =====\n";
        cout << "1. Add Server\n";
        cout << "2. View Servers\n";
```

```cpp
    int choice;

    do
    {
        cout << "\n===== SERVERPACK CLOUD SYSTEM =====\n";
        cout << "1. Add Server\n";
        cout << "2. View Servers\n";
        cout << "3. Add VM Request\n";
        cout << "4. Process VM Request\n";
        cout << "5. View VMs\n";
        cout << "6. Resize VM\n";
        cout << "7. Undo Resize\n";
        cout << "8. System Check\n";
        cout << "9. Density Sorter\n";
        cout << "10. Add Connection\n";
        cout << "11. Show Datacenter Map\n";
        cout << "12. Fastest Connection\n";
        cout << "13. Power Saver\n";
        cout << "0. Exit\n";
        cout << "Choice: ";
        cin >> choice;
        switch (choice)
        {
        case 1:
            manager.addServer();
            break;
        case 2:
            manager.viewServers();
            break;
        case 3:
            manager.addVMRequest();
            break;
        case 4:
            manager.processVMRequest();
            break;
        case 5:
            manager.viewVMs();
            break;
        case 6:
            manager.resizeVM();
            break;
        case 7:
            manager.undoResize();
            break;
        case 8:
            manager.systemCheck();
            break;
        case 9:
            manager.densitySorter();
            break;
        case 10:
            manager.addConnection();
            break;
        case 11:
            manager.showMap();
            break;
        case 12:
            manager.fastestConnection();
            break;
        case 13:
            manager.powerSaver();
            break;
        case 0:
            cout << "Thank You!\n";
            break;
        default:
            cout << "Invalid Choice.\n";
        }
    } while (choice != 0);
    return 0;
}
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

mananjain@Manans-MacBook-Air DSA(Final) % cd "/Users/mananjain/Documents/DSA(Final)/" && g++ main.cpp -o main && "/Users/mananjain
/Documents/DSA(Final)/"main
Choice: 2

Server: S3
CPU: 32
RAM: 16
VM Count: 2
-----------------

Server: S2
CPU: 32
RAM: 8
VM Count: 1
-----------------

Server: S1
CPU: 32
RAM: 8
VM Count: 1
-----------------

===== SERVERPACK CLOUD SYSTEM =====
1. Add Server
2. View Servers
3. Add VM Request
4. Process VM Request
5. View VMs
6. Resize VM
7. Undo Resize
8. System Check
9. Density Sorter
10. Add Connection
11. Show Datacenter Map
12. Fastest Connection
13. Power Saver
0. Exit
Choice: 5

VM ID: V4
OS: LINUX
RAM: 8 GB
-----------------

VM ID: V2
OS: WINDOWS
RAM: 8 GB
-----------------

VM ID: V3
OS: UBUNTU
RAM: 4 GB
-----------------

VM ID: V1
OS: MACOS
RAM: 16 GB
-----------------

===== SERVERPACK CLOUD SYSTEM =====
1. Add Server
2. View Servers
3. Add VM Request
4. Process VM Request
5. View VMs
6. Resize VM
7. Undo Resize
8. System Check
9. Density Sorter
10. Add Connection
11. Show Datacenter Map
12. Fastest Connection
13. Power Saver
0. Exit
```

```
mananjain@Manans-MacBook-Air DSA(Final) % cd "/Users/mananjain/Documents/DSA(Final)/" && g++ main.cpp -o main && "/Users/mananjain
/Documents/DSA(Final)/"main
===== SERVERPACK CLOUD SYSTEM =====
1. Add Server
2. View Servers
3. Add VM Request
4. Process VM Request
5. View VMs
6. Resize VM
7. Undo Resize
8. System Check
9. Density Sorter
10. Add Connection
11. Show Datacenter Map
12. Fastest Connection
13. Power Saver
0. Exit
Choice: 6
VM ID: V1
New RAM: 8
VM resized.

===== SERVERPACK CLOUD SYSTEM =====
1. Add Server
2. View Servers
3. Add VM Request
4. Process VM Request
5. View VMs
6. Resize VM
7. Undo Resize
8. System Check
9. Density Sorter
10. Add Connection
11. Show Datacenter Map
12. Fastest Connection
13. Power Saver
0. Exit
Choice: 7
Undo successful.

===== SERVERPACK CLOUD SYSTEM =====
1. Add Server
2. View Servers
3. Add VM Request
4. Process VM Request
5. View VMs
6. Resize VM
7. Undo Resize
8. System Check
9. Density Sorter
10. Add Connection
11. Show Datacenter Map
12. Fastest Connection
13. Power Saver
0. Exit
Choice: 8
Enter OS to verify: LINUX
OS verified.

===== SERVERPACK CLOUD SYSTEM =====
1. Add Server
2. View Servers
3. Add VM Request
4. Process VM Request
5. View VMs
6. Resize VM
7. Undo Resize
8. System Check
9. Density Sorter
10. Add Connection
11. Show Datacenter Map
12. Fastest Connection
13. Power Saver
0. Exit
```

```
0. Exit
Choice: 9

Servers by utilization:
S3 -> 2 VMs
S2 -> 1 VMs
S1 -> 1 VMs

===== SERVERPACK CLOUD SYSTEM =====
1. Add Server
2. View Servers
3. Add VM Request
4. Process VM Request
5. View VMs
6. Resize VM
7. Undo Resize
8. System Check
9. Density Sorter
10. Add Connection
11. Show Datacenter Map
12. Fastest Connection
13. Power Saver
0. Exit
Choice: 10
Server A: S1
Server B: S2
Latency: 2
Connection added.

===== SERVERPACK CLOUD SYSTEM =====
1. Add Server
2. View Servers
3. Add VM Request
4. Process VM Request
5. View VMs
6. Resize VM
7. Undo Resize
8. System Check
9. Density Sorter
10. Add Connection
11. Show Datacenter Map
12. Fastest Connection
13. Power Saver
0. Exit
Choice: 10
Server A: S2
Server B: S3
Latency: 1
Connection added.

===== SERVERPACK CLOUD SYSTEM =====
1. Add Server
2. View Servers
3. Add VM Request
4. Process VM Request
5. View VMs
6. Resize VM
7. Undo Resize
8. System Check
9. Density Sorter
10. Add Connection
11. Show Datacenter Map
12. Fastest Connection
13. Power Saver
0. Exit
Choice: 10
Server A: S1
Server B: S3
Latency: 3
Connection added.
```

```
Choice: 11

S3 -> (S2, 1ms) (S1, 3ms)
S2 -> (S1, 2ms) (S3, 1ms)
S1 -> (S2, 2ms) (S3, 3ms)

===== SERVERPACK CLOUD SYSTEM =====
1. Add Server
2. View Servers
3. Add VM Request
4. Process VM Request
5. View VMs
6. Resize VM
7. Undo Resize
8. System Check
9. Density Sorter
10. Add Connection
11. Show Datacenter Map
12. Fastest Connection
13. Power Saver
0. Exit
Choice: 12
Source Server: S1
Destination Server: S3
Minimum latency = 3 ms

===== SERVERPACK CLOUD SYSTEM =====
1. Add Server
2. View Servers
3. Add VM Request
4. Process VM Request
5. View VMs
6. Resize VM
7. Undo Resize
8. System Check
9. Density Sorter
10. Add Connection
11. Show Datacenter Map
12. Fastest Connection
13. Power Saver
0. Exit
Choice: 13

Power Saver Recommendation
Server S2 (1 VM) → move to Server S3 | Power off S2 to save power.
Server S1 (1 VM) → move to Server S3 | Power off S1 to save power.

===== SERVERPACK CLOUD SYSTEM =====
1. Add Server
2. View Servers
3. Add VM Request
4. Process VM Request
5. View VMs
6. Resize VM
7. Undo Resize
8. System Check
9. Density Sorter
10. Add Connection
11. Show Datacenter Map
12. Fastest Connection
13. Power Saver
0. Exit
```

Results and Findings

• All 8 required features were successfully implemented and tested
• unordered_map provided consistent O(1) server and VM lookups — significantly faster
than linear search
• Stack-based undo proved reliable for both single and multiple sequential resize rollbacks

• The provisioning queue maintained strict FIFO order across all test cases
• Binary search on the sorted OS list correctly verified valid and invalid operating systems
• Dijkstra's algorithm correctly computed minimum latency paths across multi-hop server connections
• The density sorter successfully ranked servers from most to least utilized
• The power saver correctly identified underloaded servers and recommended consolidation targets
• The system architecture mirrors real-world platforms — VMware vCenter (inventory), OpenStack Nova (queue), OpenStack Neutron (network graph)

Conclusion

The ServerPack Cloud Tenant Virtual Machine Allocator successfully demonstrates the practical application of core data structures — hash maps, stacks, queues, graphs, and heaps — in solving real-world datacenter management challenges.

Each data structure was chosen deliberately based on its time and space efficiency for the
specific use case it serves. The system is modular, readable, and extensible, and reflects the
architectural patterns found in industry-grade platforms like VMware and OpenStack.

GitHub Repository

The complete source code, README, and screenshots are available at:
https://github.com/mananjain20/virtual-machine-allocator-dsa